

**Modern Education Society's
Wadia College of Engineering, Pune-01**

NAME OF STUDENT:	CLASS:
SEMESTER/YEAR:	ROLL NO:
DATE OF PERFORMANCE:	DATE OF SUBMISSION:
EXAMINED BY:	EXPERIMENT NO:

TITLE: ASSIGNMENT ON Mini Project.

Instructions:- Add the following in the mini project report:

1. Mini Project Title
2. Problem Statement
3. Objectives
4. Hardware and Software Requirements
5. Dataset Description
6. Methodology / Algorithm
7. Practical Implementation
8. Conclusion

TweetLens: A Litmus Test for Tweets

Twitter Sentiment Analysis with Machine Learning: Mini Project Report

1. Mini Project Title

TweetLens: Explainable Multi-Model Sentiment Analysis of Tweets.

TweetLens is a full-stack machine learning web application that classifies the sentiment of a tweet as *negative*, *neutral* or *positive*. It compares four different models side by side, shows step by step how a tweet is cleaned before it reaches the models, and explains which words pushed each prediction. The interface uses a litmus-paper metaphor: every tweet is "dipped" and the strip turns red (negative), violet (neutral) or blue (positive).

Live deployment: frontend on Netlify (<https://tweetlens.netlify.app>), API on Render (<https://tweetlens-api.onrender.com>).

2. Problem Statement

Millions of tweets are posted every day about products, services, events and public figures. Reading them manually to understand public opinion is impossible at that scale, so organisations need automatic sentiment analysis. Tweets are, however, much harder to analyse than ordinary text: they are short (at most 280 characters), informal and noisy. They contain hashtags, @mentions, links, emojis, slang, deliberate misspellings ("soooo goood"), negation ("not bad at all"), sarcasm, and, in India, code-mixed Hinglish ("movie ekdum bakwas thi").

The problem addressed in this project is: *build and evaluate machine learning models that classify the sentiment of a tweet into three classes, make the models' behaviour transparent to the user, and test where they fail*, all packaged as an interactive web application that can analyse a single tweet, compare models, stream tweets and process a whole CSV file of tweets.

3. Objectives

1. To design a text preprocessing pipeline tailored to tweets: removing links and mentions, converting emojis to words, splitting hashtags, expanding contractions, translating common Hinglish words, handling negation and removing stopwords.
2. To train and compare four sentiment classifiers of different kinds: a lexicon-based model (VADER), Multinomial Naive Bayes, Logistic Regression with TF-IDF features, and a neural network (Multi-Layer Perceptron).
3. To evaluate the models fairly on a held-out benchmark using accuracy, macro F1 score, macro recall and confusion matrices.
4. To make predictions explainable by showing how much each word contributed to the score (leave-one-out explanation).

5. To additionally detect emotion (anger, joy, optimism, sadness) and irony in a tweet.
6. To test model robustness on hand-written difficult tweets (negation, sarcasm, Hinglish, emoji-only, slang).
7. To build an accessible, responsive web interface and deploy the complete system to the cloud.

4. Hardware and Software Requirements

4.1 Hardware Requirements

Component	Minimum	Used for development
Processor	Dual-core 64-bit CPU	4-core 64-bit CPU
RAM	4 GB (the trained server needs about 220 MB)	16 GB
Storage	2 GB free (dependencies, dataset and models)	SSD
GPU	Not required: all models train on a CPU in about 2 to 3 minutes	
Network	Internet connection to download the dataset and packages	

4.2 Software Requirements

Category	Software / Library	Purpose
Operating system	Linux, Windows or macOS	Development and hosting
Language (backend)	Python 3.11	Machine learning and API
Machine learning	scikit-learn 1.9, NumPy 2.4, pandas 3.0, joblib	Feature extraction, models, evaluation
NLP	vaderSentiment 3.3, emoji 2.16, wordsegment 1.3	Lexicon model, emoji names, hashtag splitting
Web API	FastAPI 0.142, Uvicorn 0.54	REST API and live stream (server-sent events)
Language (frontend)	TypeScript, React 19, Vite 8	User interface
UI libraries	framer-motion, rough-notation, D3, culori	Animation, word underlines, charts, colours
Testing	pytest, Vitest, Playwright	Unit tests and screenshots
Tools	Git, GitHub, VS Code	Version control and editing
Deployment	Render (API), Netlify (web app)	Cloud hosting

5. Dataset Description

5.1 TweetEval Sentiment (main dataset)

The models are trained on the **TweetEval** benchmark (Barbieri et al., 2020), sentiment task, which comes from SemEval-2017 Task 4A. Each tweet is labelled by human annotators as 0 = negative, 1 = neutral or 2 = positive. The dataset has fixed train, validation and test splits, so results are comparable with published work. Usernames are anonymised as @user.

Split	Tweets	Negative	Neutral	Positive
Train	45,615	15.5%	45.3%	39.1%
Validation	2,000	15.6%	43.4%	40.9%
Test	12,284	3,972 (32.3%)	5,937 (48.3%)	2,375 (19.3%)

Table 1: TweetEval sentiment splits and class distribution

The test split has twice the share of negative tweets that the training split has. This class shift is why the models use balanced class weights and why macro-averaged metrics are reported.

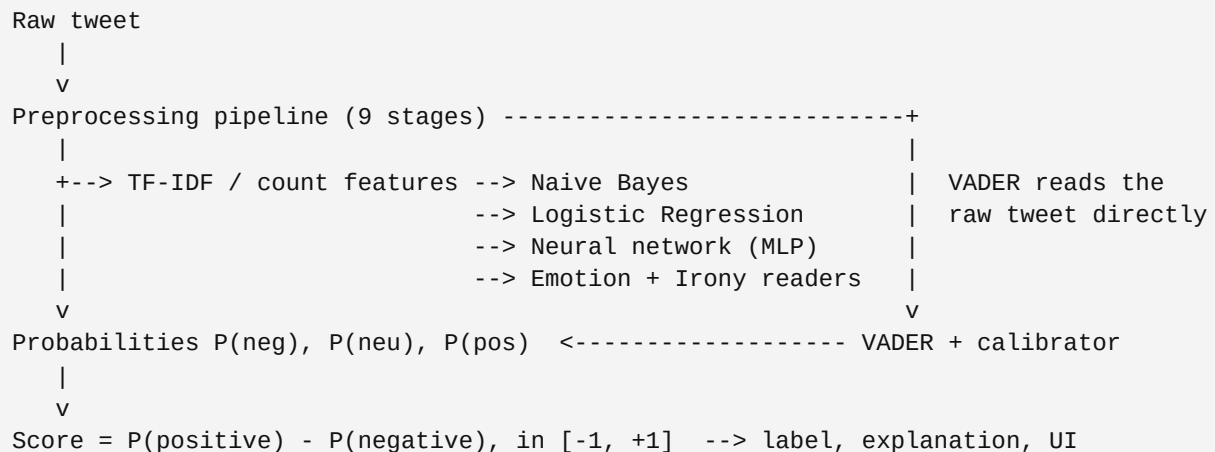
5.2 Additional datasets

Dataset	Size	Labels	Used for
TweetEval Emotion (SemEval-2018)	3,257 train / 1,421 test	anger, joy, optimism, sadness	Emotion reader
TweetEval Irony (SemEval-2018)	2,862 train / 784 test	ironic, not ironic	Irony reader
Twitter US Airline Sentiment (Feb 2015)	1,200-tweet sample	negative, neutral, positive + date	Bulk analyzer demo (out-of-domain test)
Robustness suite (written for this project)	36 tweets, 9 categories	expected label + reason	Robustness lab

6. Methodology / Algorithm

6.1 System architecture

The system has two parts. The **backend** (Python, FastAPI) holds the preprocessing pipeline, the trained models and the REST API. The **frontend** (React, TypeScript) is the web interface that calls the API. The models are trained once by a training script and saved with joblib; the API loads them at start-up.



6.2 Preprocessing algorithm

Every tweet passes through nine stages. Each stage records which tokens it removed, rewrote, split or fused, so the interface can show the transformation step by step.

1. **Tokenise** the raw tweet, keeping URLs, @mentions, #hashtags, emojis and emoticons whole.
2. **Remove** links, mentions and retweet marks (they carry no sentiment).
3. **Convert emojis and emoticons to words**: 😡 → enraged_face, :(→ frowning_face.
4. **Split hashtags** on capitals or with a word-frequency model: #NeverAgain → never again.
5. **Lowercase** and squeeze stretched words: SOOOO → soo, !!! → !.
6. **Expand contractions**: don't → do not, cant → can not.
7. **Translate Hinglish words** (58-word lexicon): bakwas → rubbish, achha → good, nahi → not.
8. **Fuse negation**: a negator joins the next content words until punctuation or "but": not happy → NOT_happy. Hindi "nahi" comes after the word, so it fuses backwards: achhi nahi → NOT_good.
9. **Remove stopwords** and most punctuation (! and ? are kept because they carry tone).

6.3 Feature extraction

- **Count vectors** (word unigrams + bigrams, min document frequency 2) for Naive Bayes.
- **TF-IDF** (term frequency × inverse document frequency, sublinear TF) of word unigrams and bigrams, combined with character 2–5-grams (up to 300,000 features) for Logistic Regression. Character n-grams make the model robust to typos and stretched spellings.

- TF-IDF of word unigrams and bigrams (30,000 features) for the neural network.

$tf-idf(t, d) = (1 + \ln tf(t, d)) \times (\ln((1 + N) / (1 + df(t))) + 1)$, where N is the number of tweets and $df(t)$ the number of tweets containing t

6.4 Models

Model	Algorithm	Key settings
VADER	Rule- and lexicon-based scorer (about 7,500 rated words, rules for negation, capitals, "but" and exclamation marks), followed by a small logistic regression that calibrates its four scores into three probabilities	Balanced class weights
Naive Bayes	Multinomial Naive Bayes: $P(\text{class} \text{words}) \propto P(\text{class}) \cdot \prod P(\text{word} \text{class})$	Laplace smoothing $\alpha = 1.0$
Logistic Regression	Multinomial (softmax) logistic regression on TF-IDF features	$C = 1.0$, balanced class weights, L-BFGS
Neural net (MLP)	One hidden layer of 128 ReLU units, softmax output, Adam optimiser	L2 $\alpha = 0.01$, early stopping, balanced sample weights

Hyperparameters were chosen on the validation split (for example, $C \in \{1, 3\}$ and balanced or unbalanced weights for logistic regression). The emotion and irony readers are logistic regressions using the same pipeline and features.

6.5 Scoring and explanation

Each model outputs three probabilities. The displayed **score** = **P(positive)** – **P(negative)**, from -1 to $+1$, and the predicted label is the class with the highest probability. For **explainability**, the leave-one-out method is used: each word is removed in turn, the tweet is scored again, and the word's weight is the change in score. Positive weights pushed the prediction towards positive, negative weights towards negative.

6.6 Evaluation metrics

- **Accuracy** = correct predictions / total predictions.
- **Precision** = $TP / (TP + FP)$, **Recall** = $TP / (TP + FN)$, **F1** = $2 \cdot P \cdot R / (P + R)$, computed per class.
- **Macro F1** and **macro recall** = the unweighted mean over the three classes, so the large neutral class cannot hide a weak negative class. Macro recall is the official TweetEval metric.
- **Confusion matrix**: rows are the true labels, columns the predicted labels.

7. Practical Implementation

7.1 Key code

Negation handling in the preprocessing pipeline (backend/tweetlens/preprocess.py, simplified):

```
NEGATORS = {"not", "no", "never", "nothing", "nobody", "none", "nor", "cannot", ...}

def _stage_negation(tokens, scope=3):
    for i, t in enumerate(tokens):
        if t.text not in NEGATORS:
            continue
        if t.post_negator:          # Hinglish "nahi" negates the word BEFORE it
            targets = [previous content word]
        else:                      # English: next content words until , . ! ?
            or "but"
            targets = next content words (max 3)
        for k in targets:
            tokens[k].text = "NOT_" + tokens[k].text
            drop the negator token          # e.g. "not good" -> "NOT_good"
```

Training the logistic regression model (backend/tweetlens/train.py):

```
features = FeatureUnion([
    ("word", TfidfVectorizer(token_pattern=r"\S+", ngram_range=(1, 2), min_df=2,
        sublinear_tf=True)),
    ("char", TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 5), min_df=3,
        sublinear_tf=True, max_features=300_000)),
])
logreg = Pipeline([("features", features),
    ("clf", LogisticRegression(C=1.0, max_iter=3000,
        class_weight="balanced"))])
logreg.fit(clean_many(x_train), y_train)
joblib.dump(logreg, ARTIFACTS_DIR / "logreg.joblib", compress=3)
```

Output of the training script (python -m tweetlens.train):

```
Reading TweetEval sentiment
  pipeline over 59,899 tweets in 13s
Calibrating VADER
Training Naive Bayes
Training logistic regression
Training the neural net
Training the emotion reader
  emotion: macro F1 0.684
Training the irony reader
  irony: macro F1 0.687
Scoring the test split
  VADER          accuracy 0.554  macro F1 0.550  macro recall 0.582
  Naive Bayes    accuracy 0.591  macro F1 0.585  macro recall 0.595
  Logistic regression accuracy 0.624  macro F1 0.623  macro recall 0.635
  Neural net     accuracy 0.560  macro F1 0.564  macro recall 0.589
Done in 129s. Artifacts are in backend/artifacts
```

7.2 REST API

Endpoint	Method	Function
/api/analyze	POST	All four readings, the 9-stage pipeline trace, emotions and irony for one tweet
/api/explain	POST	Leave-one-out word weights for a chosen model
/api/robustness	GET	Scores the 36-tweet robustness suite with every model
/api/pulse	GET	Server-sent event stream that replays the test split as live tweets
/api/bulk	POST	Reads an uploaded CSV of tweets and returns per-tweet and summary results
/api/meta	GET	Model descriptions and test metrics

7.3 Results on the test set (12,284 tweets)

Model	Accuracy	Macro F1	Macro recall	F1 neg	F1 neu	F1 pos	Time / tweet
VADER (calibrated)	0.554	0.550	0.582	0.600	0.554	0.497	0.17 ms
Naive Bayes	0.591	0.585	0.595	0.569	0.612	0.573	1.02 ms
Logistic Regression	0.624	0.623	0.635	0.635	0.621	0.614	1.93 ms
Neural net (MLP)	0.560	0.564	0.589	0.593	0.513	0.586	0.87 ms
Baseline: always "neutral"	0.483	0.217	0.333	0	0.651	0	–

Table 2: Test-set performance. Logistic Regression is the best model and is used by default.

Logistic regression reaches 63.5% macro recall, slightly above the 62.9% reported for the SVM and FastText baselines in the TweetEval paper. All models clearly beat the majority-class baseline.

True label	Predicted by Logistic Regression		
	negative	neutral	positive
negative	2,728	1,013	231
neutral	1,687	3,416	834
positive	211	639	1,525

Table 3: Confusion matrix of the best model. Most errors are between neutral and its neighbours, rarely between negative and positive.

7.4 Application screenshots

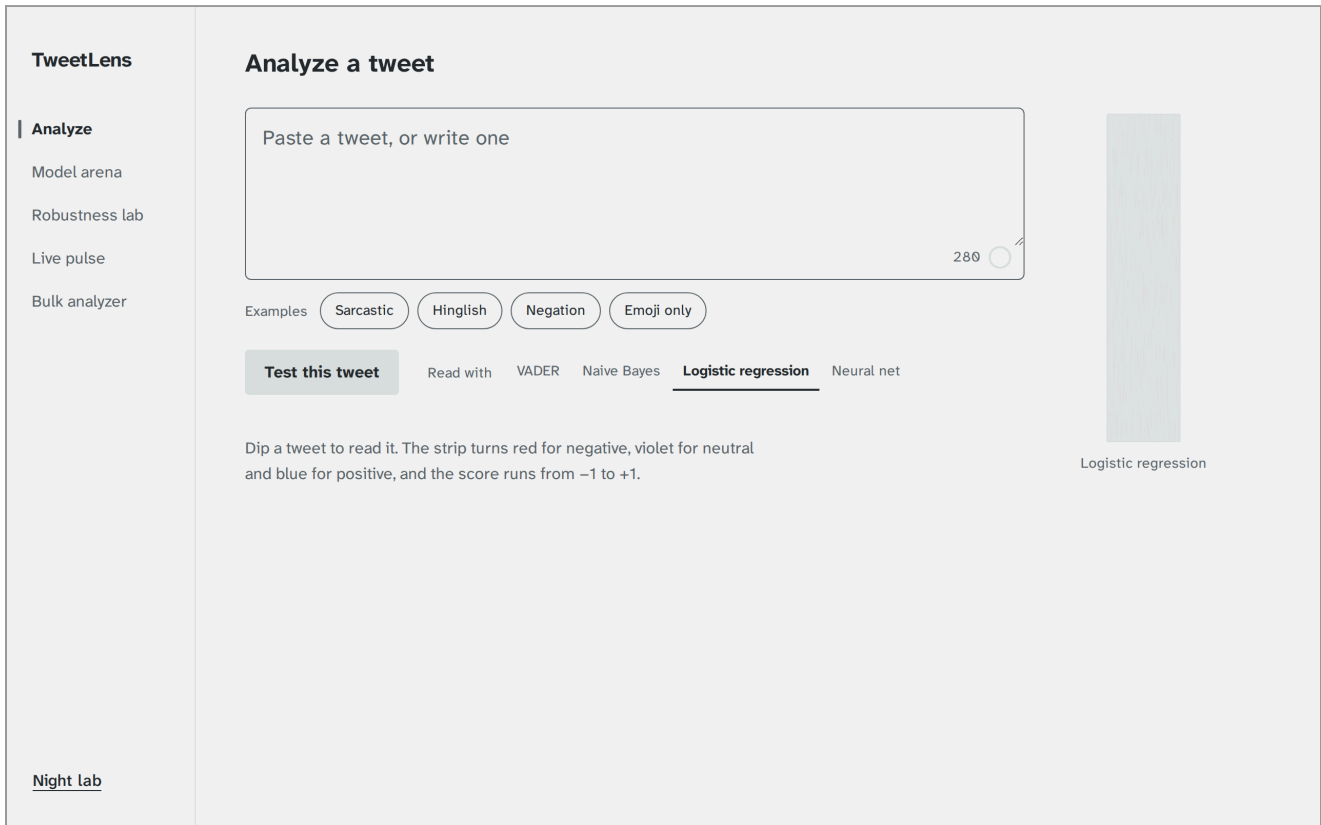


Figure 1: Home page (Analyze). The composer, example chips and the dry litmus strip.

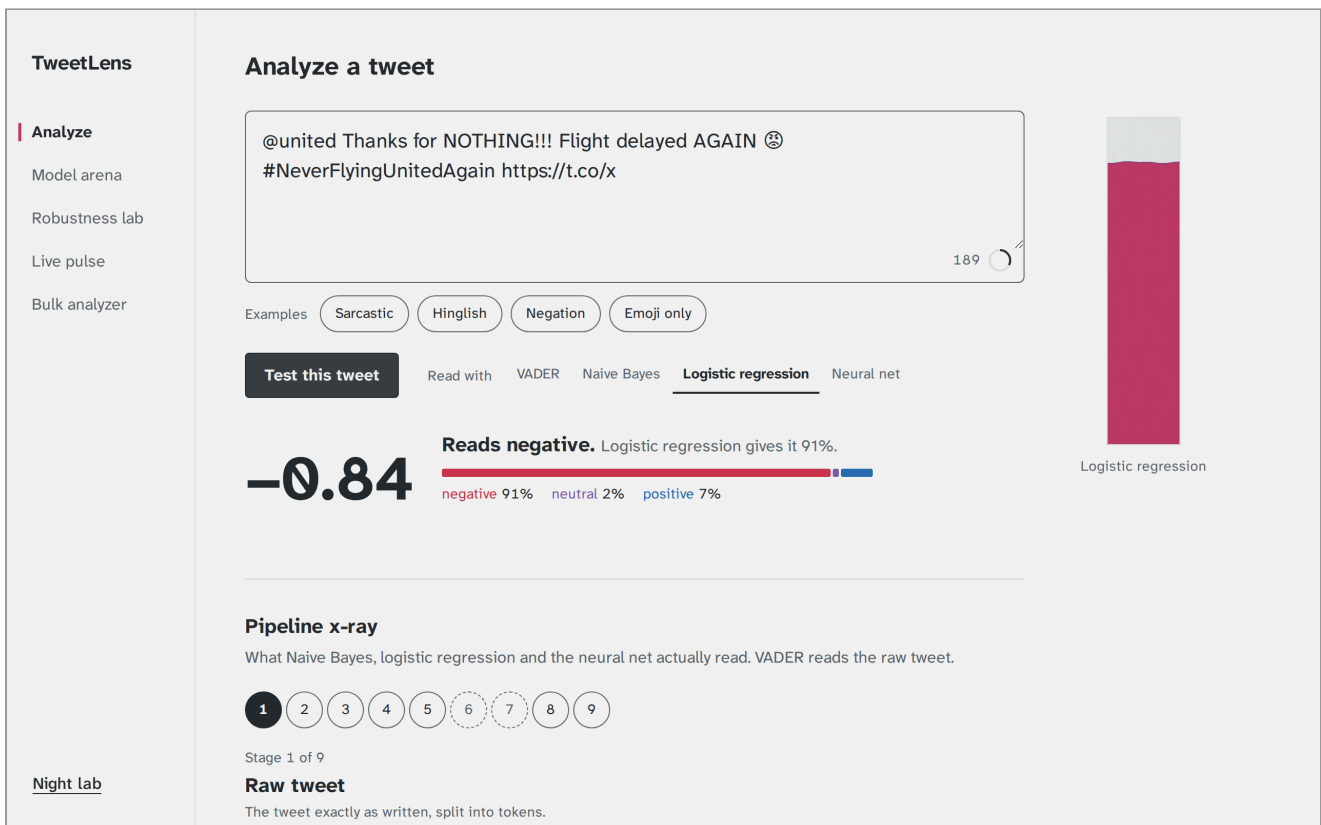


Figure 2: Analysing a complaint tweet. Logistic regression scores it -0.84 (91% negative) and the strip turns red.

Pipeline x-ray

What Naive Bayes, logistic regression and the neural net actually read. VADER reads the raw tweet.

1 2 3 4 5 6 7 8 9

Stage 1 of 9

Raw tweet

The tweet exactly as written, split into tokens.

@united Thanks for NOTHING !!! Flight delayed AGAIN 😡

#NeverFlyingUnitedAgain https://t.co/x

11 tokens. Step forward to watch the tweet turn into what the models read.

Previous stage Next stage Play all or use the arrow keys

Figure 3: Pipeline X-Ray, stage 1: the raw tweet split into tokens.

Pipeline x-ray

What Naive Bayes, logistic regression and the neural net actually read. VADER reads the raw tweet.

1 2 3 4 5 6 7 8 9

Stage 4 of 9

Hashtags split apart

#NeverAgain becomes never and again. Capitals mark the breaks; otherwise a word-frequency model finds them.

Thanks for NOTHING !!! Flight delayed AGAIN enraged_face Never Flying United Again

1 token split apart.

Previous stage Next stage Play all or use the arrow keys

Figure 4: Stage 4: the hashtag #NeverFlyingUnitedAgain is split into words (changed tokens highlighted).

Pipeline x-ray

What Naive Bayes, logistic regression and the neural net actually read. VADER reads the raw tweet.

1 2 3 4 5 6 7 8 9

Stage 8 of 9

Negation fused

A negator joins the words it governs, up to the next punctuation or but: not happy becomes NOT_happy, a different word from happy.

thanks for nothing ! flight delayed again enragèd_face **NOT_flying**
NOT_united again

1 negation fused, 1 token rewritten.

[Previous stage](#) [Next stage](#) [Play all](#) or use the arrow keys

Figure 5: Stage 8: negation fused. "never flying united" becomes NOT_flying NOT_united.

Pipeline x-ray

What Naive Bayes, logistic regression and the neural net actually read. VADER reads the raw tweet.

1 2 3 4 5 6 7 8 9

Stage 9 of 9

Stopwords and most punctuation removed

Words like the, is and of carry little tone and are dropped. ! and ? stay, because they do carry tone.

thanks nothing ! flight delayed enragèd_face **NOT_flying** **NOT_united**

3 tokens removed.

[Previous stage](#) [Next stage](#) [Play again](#) or use the arrow keys

Figure 6: Stage 9: stopwords removed. These are the final tokens the models read.

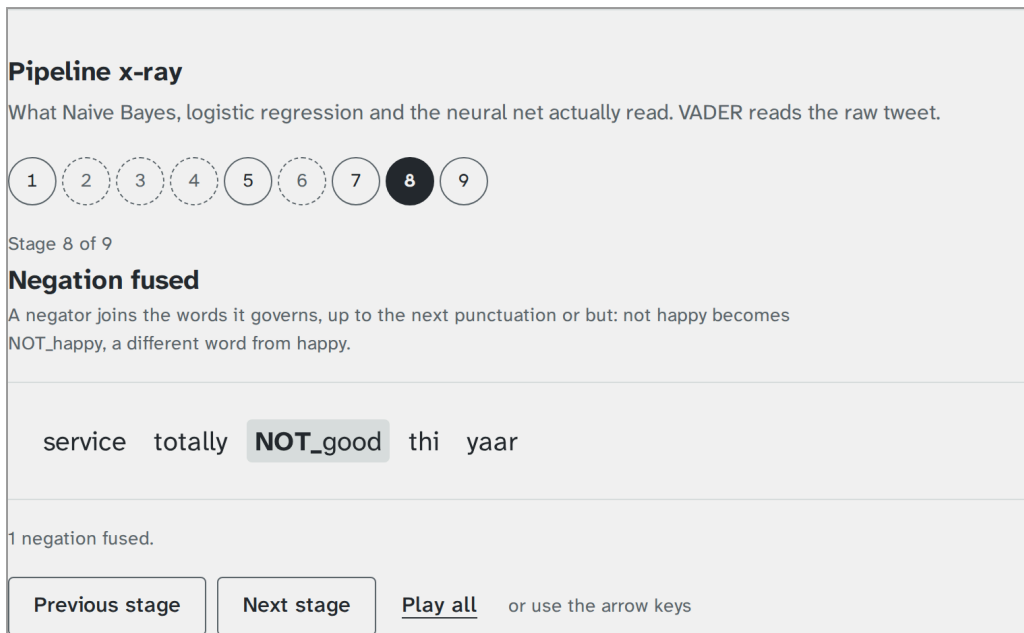


Figure 7: Hinglish handling: "achhi nahi" (good not) is translated and fused backwards into NOT_good.

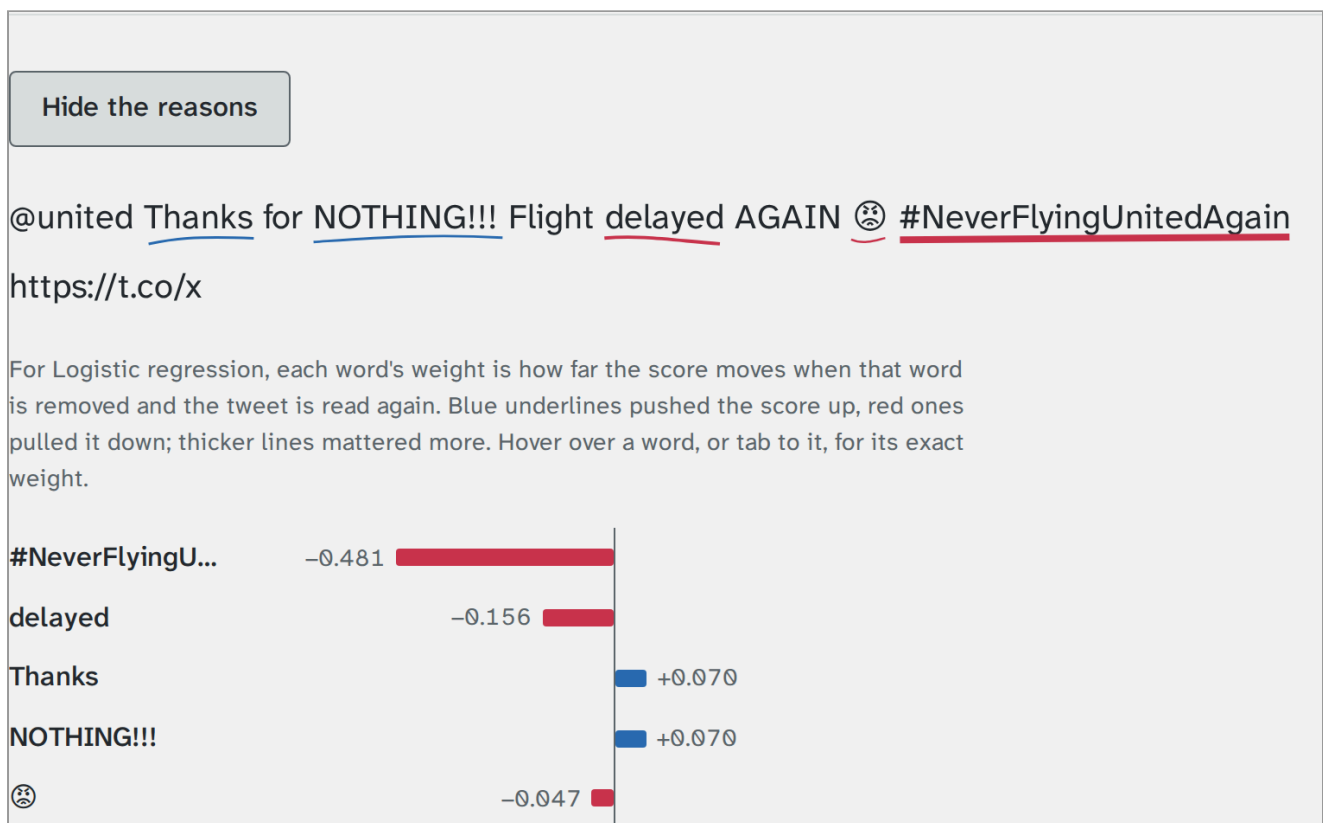


Figure 8: "Why this prediction?" Red underlines pulled the score down, blue pushed it up; the chart lists the word weights.

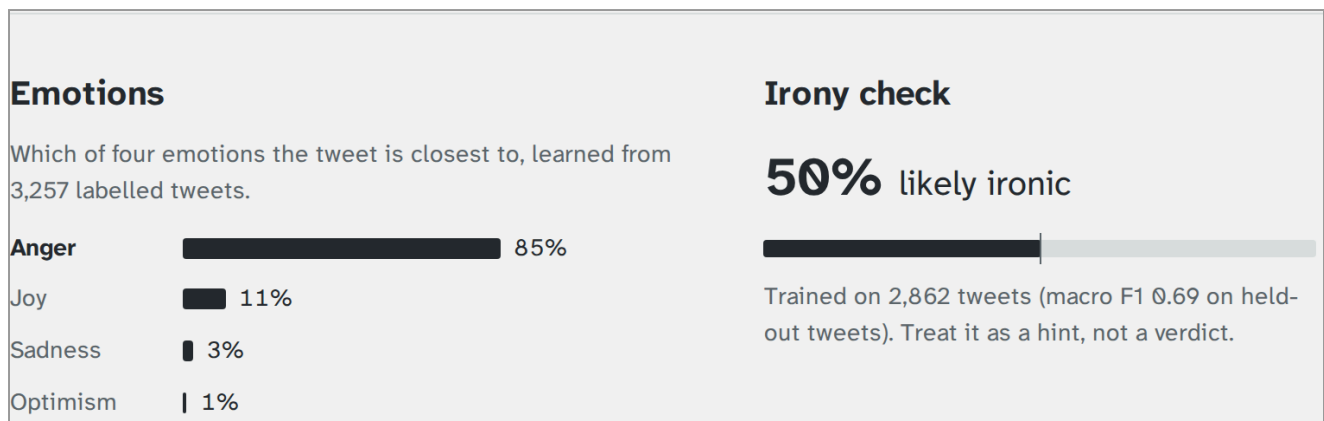


Figure 9: Emotion probabilities and the irony check for the same tweet.

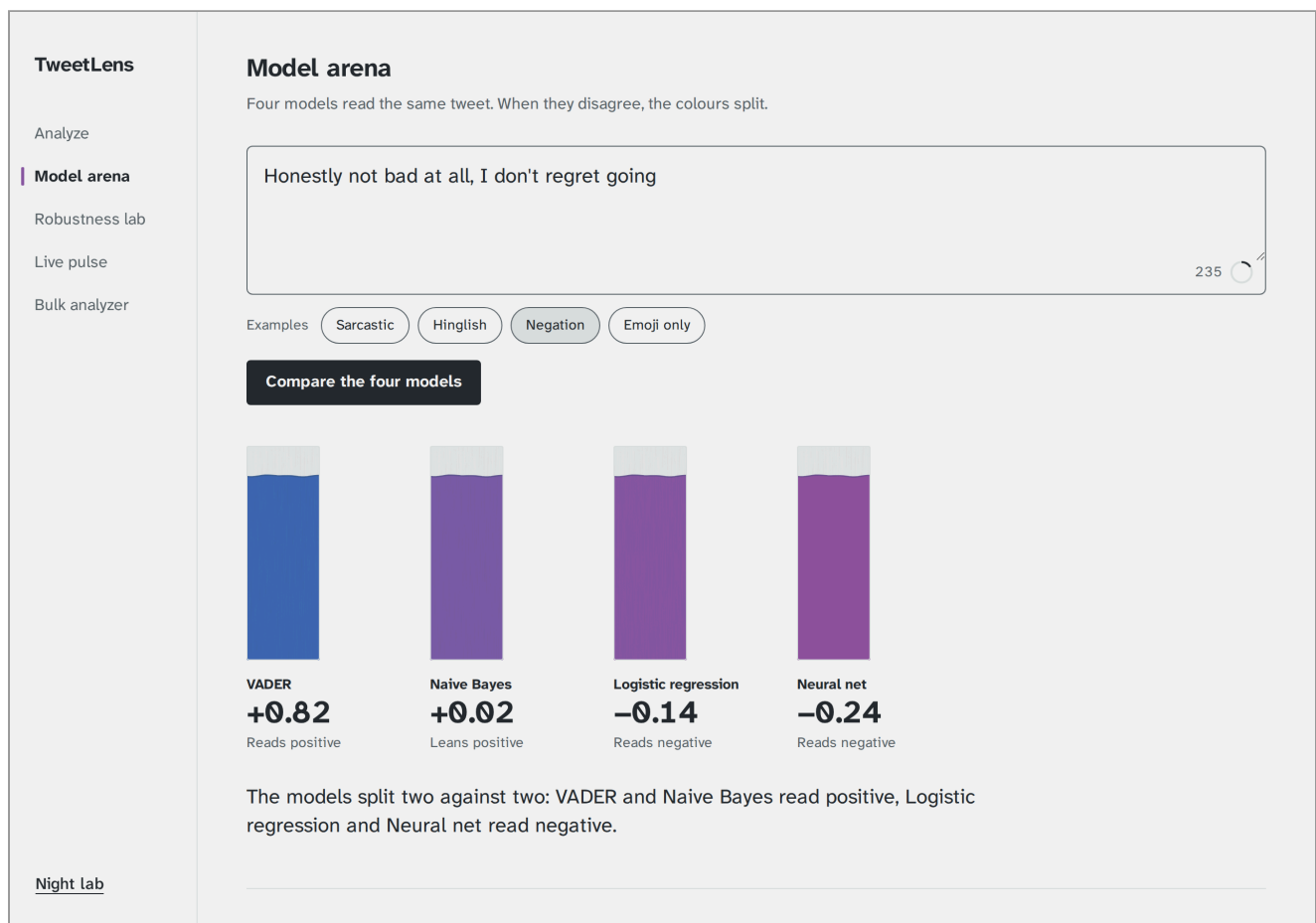


Figure 10: Model Arena. The four models read "Honestly not bad at all, I don't regret going" and split two against two.

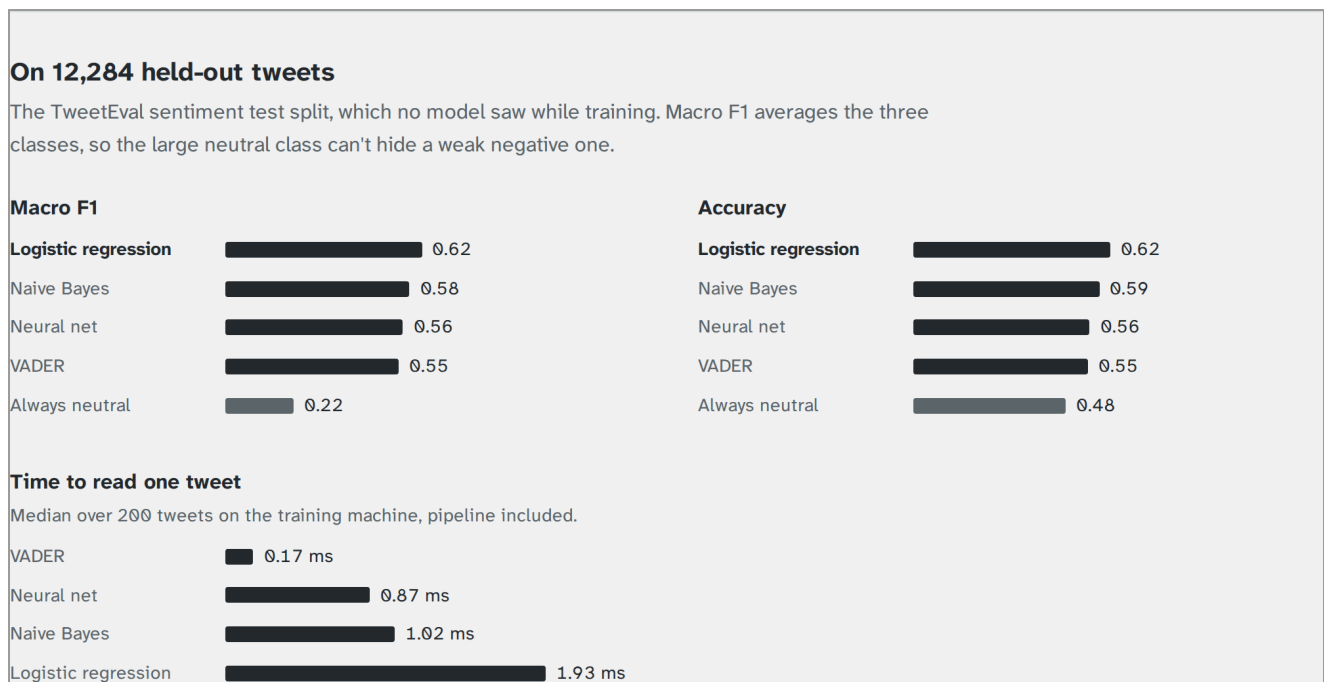


Figure 11: Benchmark on the 12,284 held-out tweets: macro F1, accuracy and speed of each model.

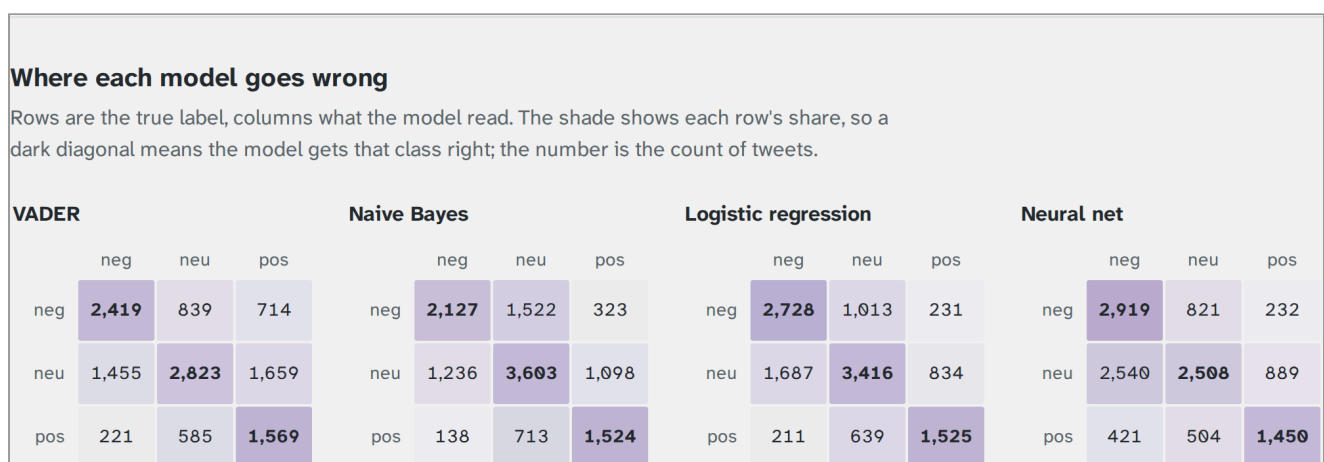
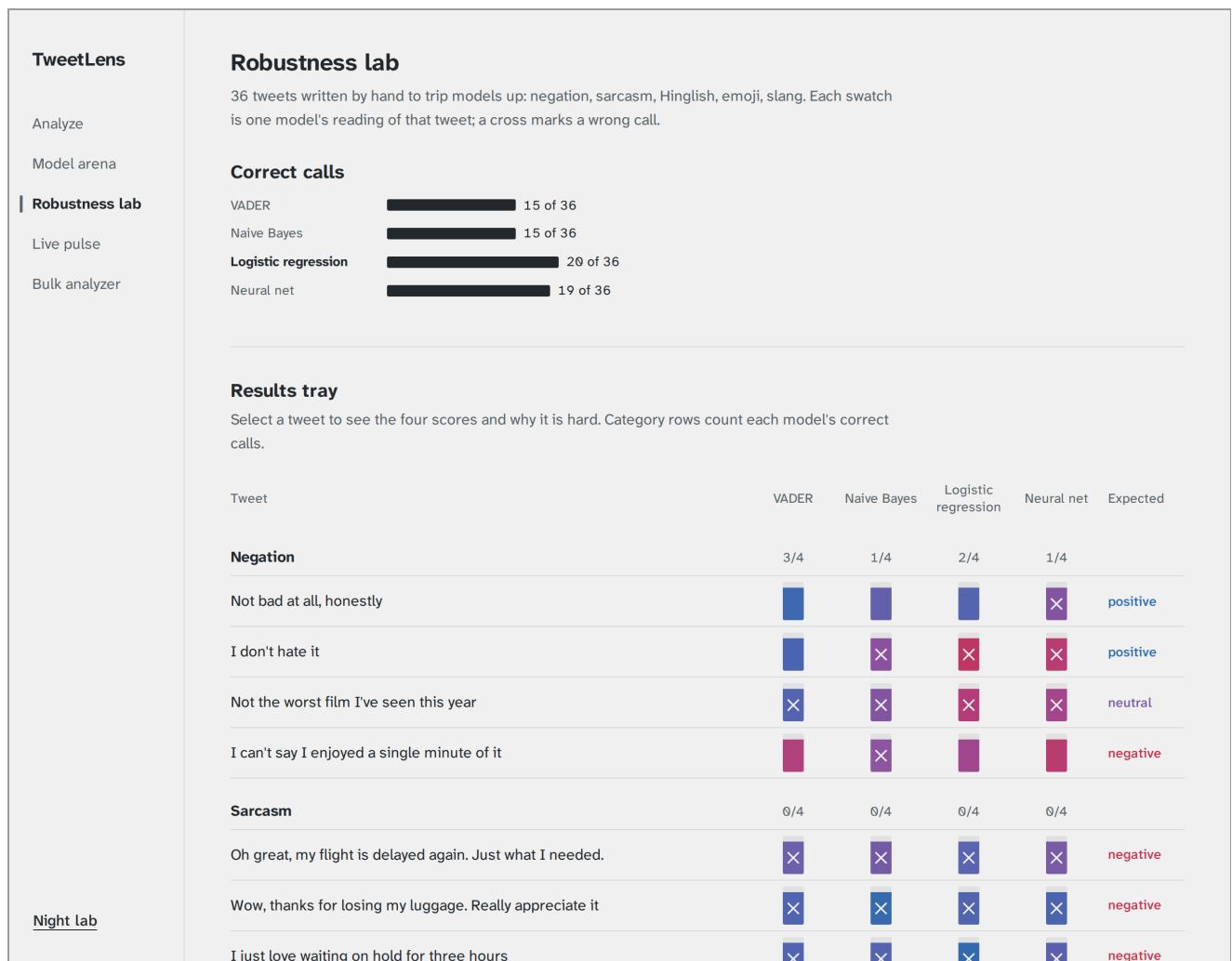


Figure 12: Confusion matrices of all four models (rows: true label, columns: prediction).



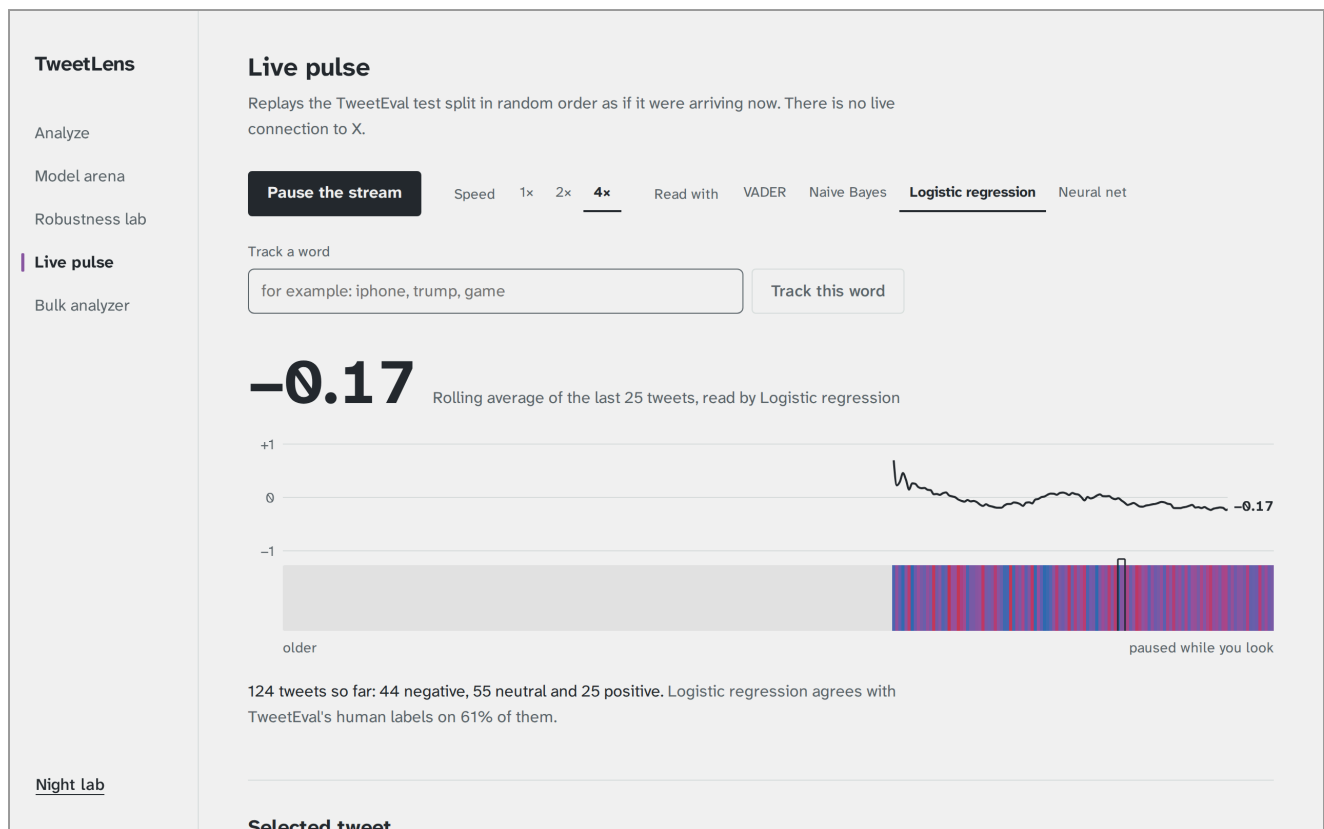


Figure 14: Live Pulse. Tweets replayed as a stream; each band is one tweet and the line is the rolling average of the last 25.

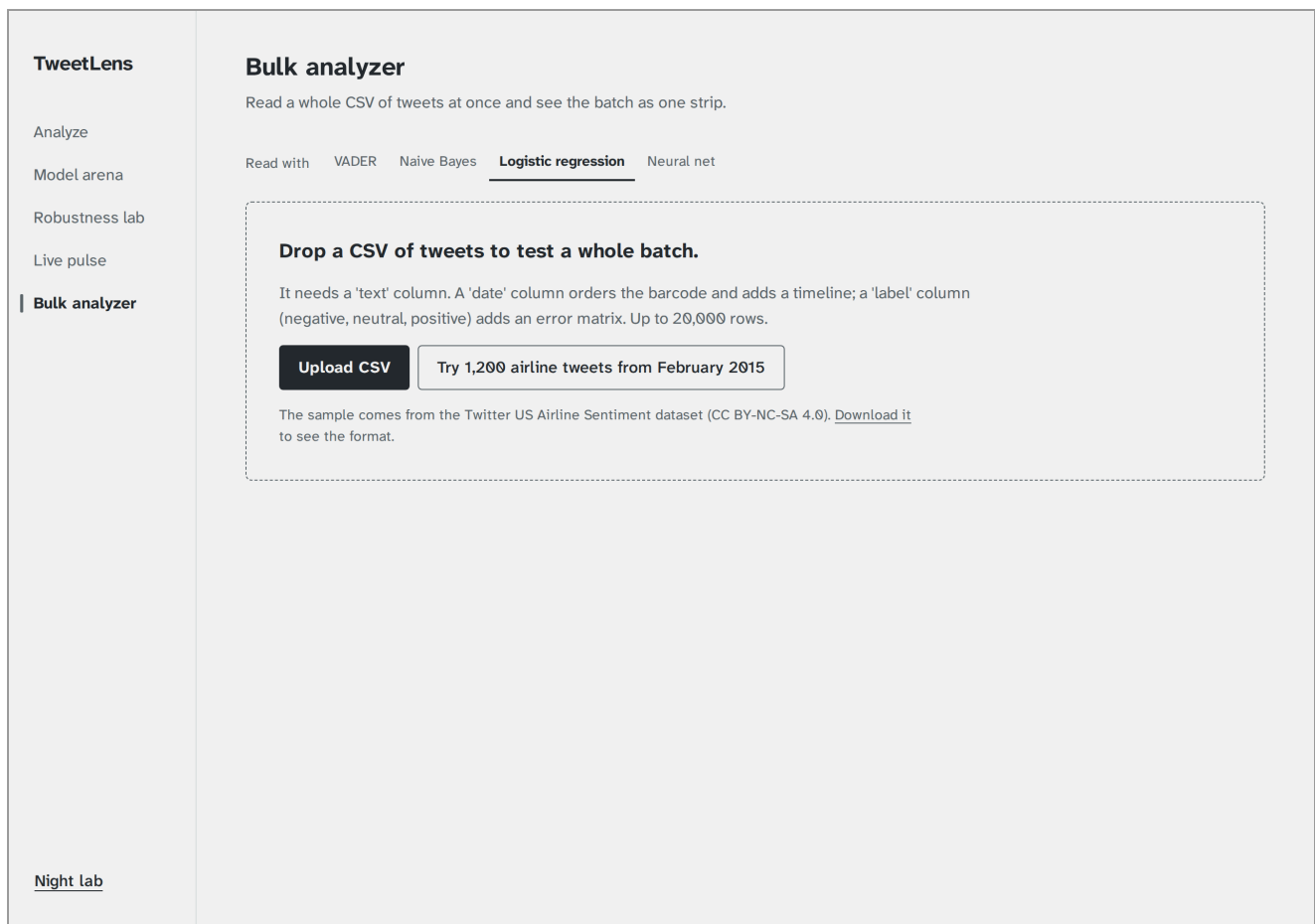


Figure 15: Bulk Analyzer, upload screen for a CSV file of tweets.

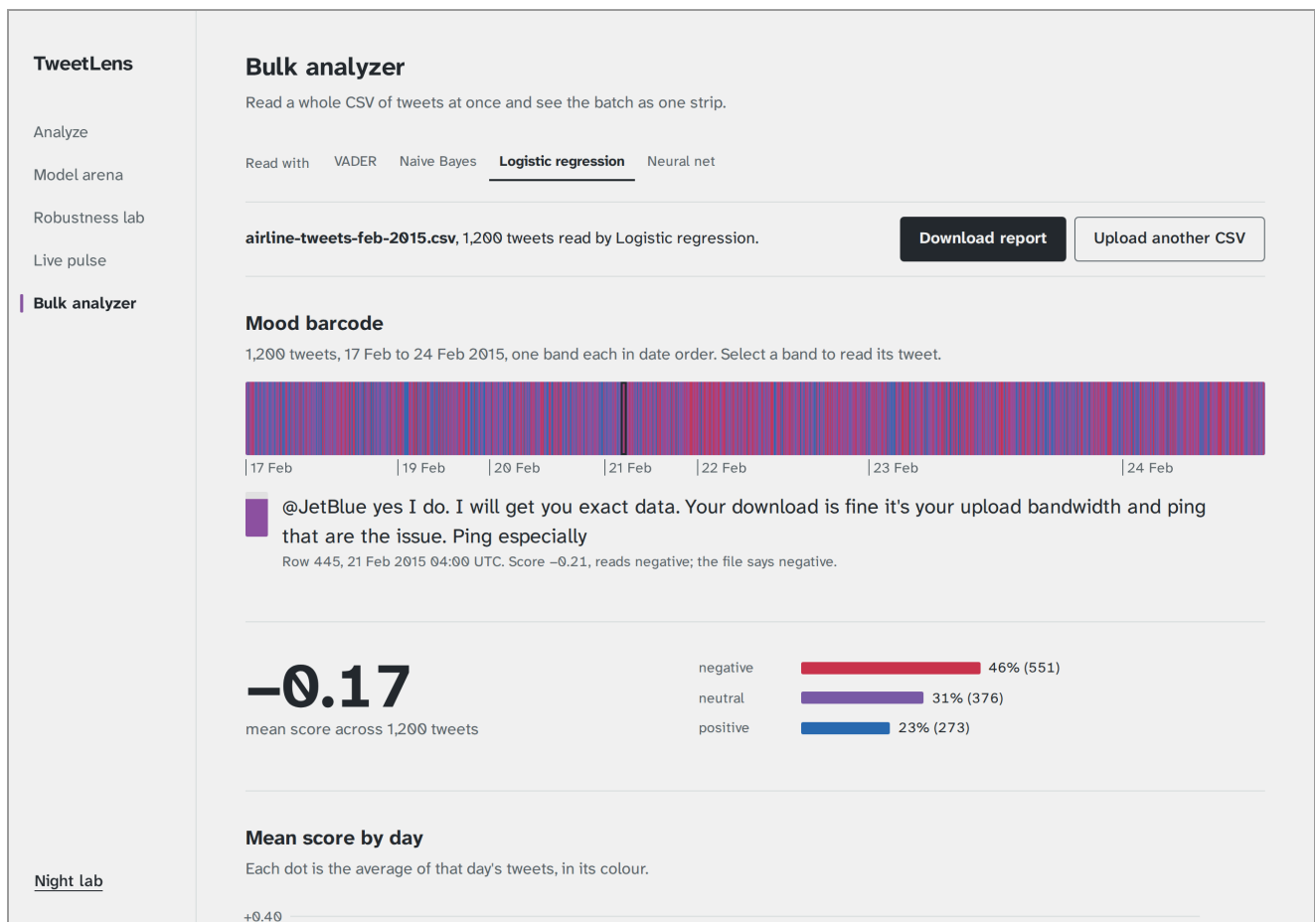


Figure 16: Bulk result for 1,200 airline tweets: the "mood barcode" (one band per tweet in date order), a selected tweet and the share of each class.

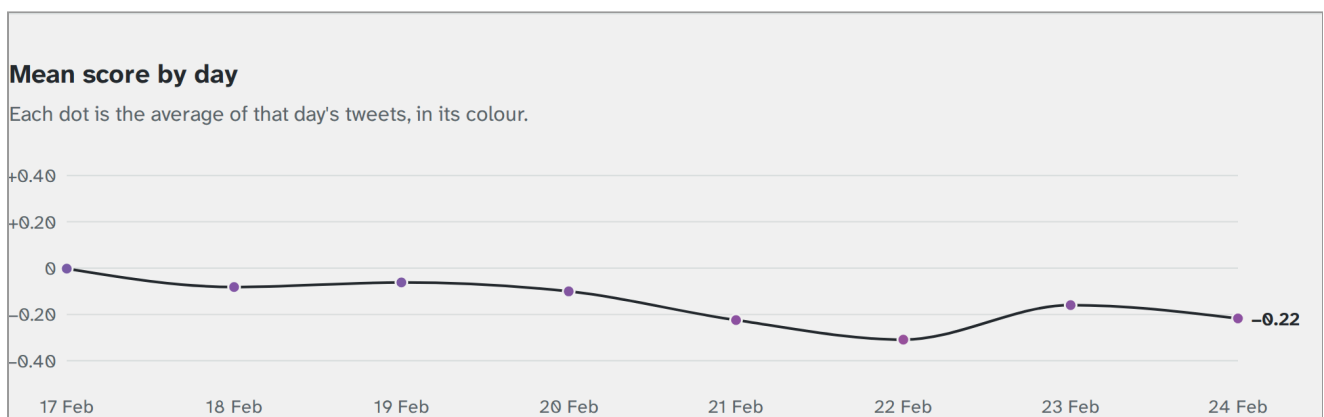


Figure 17: Mean sentiment score per day for the airline tweets.

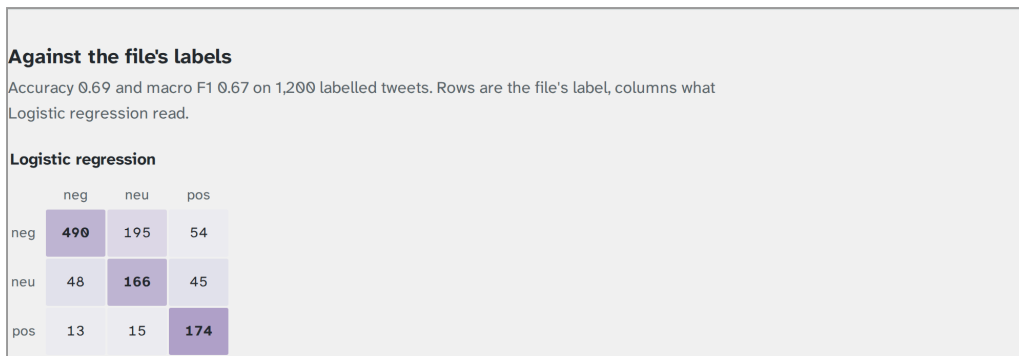


Figure 18: Out-of-domain check against the airline dataset's labels: accuracy 0.69, macro F1 0.67.

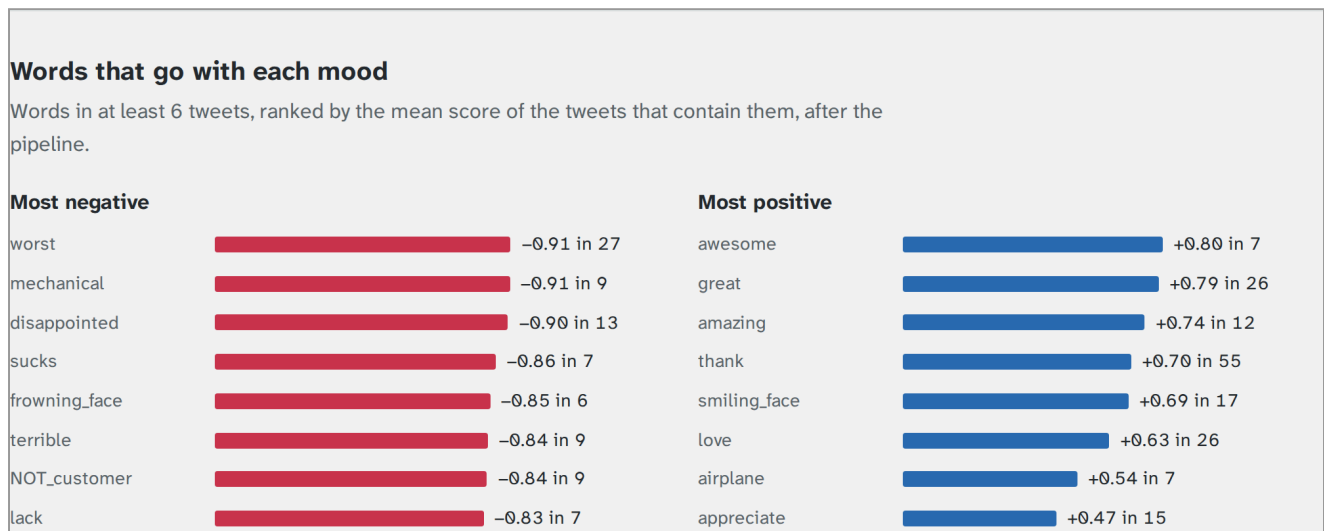


Figure 19: Words most associated with negative and positive tweets in the uploaded batch.

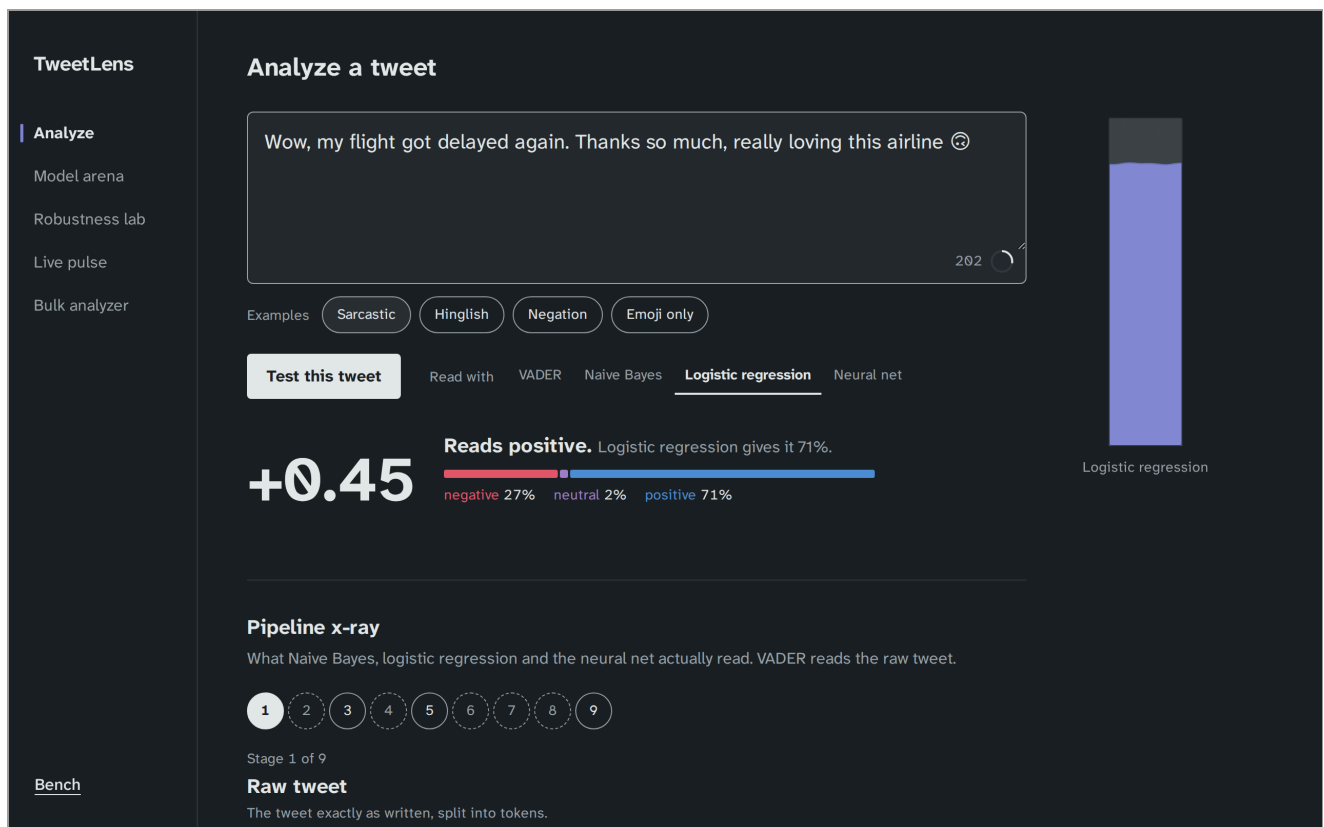


Figure 20: Dark theme ("Night lab"). A sarcastic tweet fools the model, but the irony check flags it.

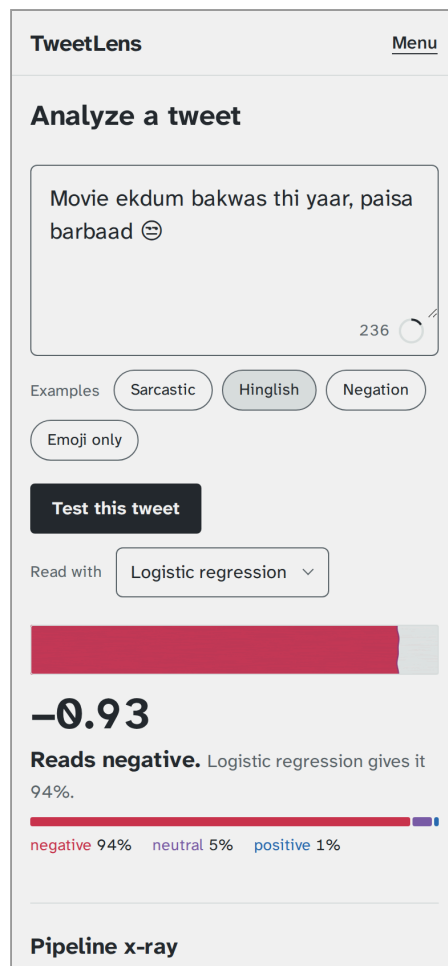


Figure 21: Responsive mobile layout (390 px wide); the strip becomes a horizontal band.

7.5 Testing and deployment

The backend has 33 automated tests (pytest) covering every preprocessing stage, CSV handling and all API endpoints; the frontend has 11 tests (Vitest) for colour contrast and formatting. All pass. The API is deployed on Render, which installs the dependencies and trains the models during each build (about 8 minutes, giving the same scores as above), and the web app is deployed on Netlify.

8. Conclusion

TweetLens shows that classical machine learning, combined with preprocessing designed for tweets, gives a useful and fully explainable sentiment analyser. Among the four models, **TF-IDF with Logistic Regression performed best** (accuracy 62.4%, macro F1 0.623, macro recall 63.5%), slightly above the published bag-of-words baselines on the same benchmark, while answering in about 2 milliseconds per tweet. Naive Bayes and the neural network were close behind, and the lexicon-based VADER was weakest on this data but fastest.

The project also showed clearly where these models fail. Tweet-specific preprocessing helped: hashtag splitting and the Hinglish stage let the trained models handle all hashtag and Hinglish test tweets. Sarcasm, however, was misread by every model, which is why a separate irony detector is shown as a

warning. The leave-one-out explanations, the step-by-step pipeline view and the side-by-side model comparison make the system transparent instead of a black box.

Future scope: fine-tuning a transformer model such as Twitter-RoBERTa (published scores above 71% macro recall), a larger Hinglish vocabulary or a code-mixed dataset, better sarcasm detection, and connecting a real Twitter/X data stream in place of the replayed test set.

References

1. F. Barbieri, J. Camacho-Collados, L. Espinosa-Anke, L. Neves. "TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification." Findings of EMNLP, 2020.
2. S. Rosenthal, N. Farra, P. Nakov. "SemEval-2017 Task 4: Sentiment Analysis in Twitter." SemEval, 2017.
3. C. J. Hutto, E. Gilbert. "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text." ICWSM, 2014.
4. F. Pedregosa et al. "Scikit-learn: Machine Learning in Python." JMLR 12, 2011.
5. Twitter US Airline Sentiment dataset, Figure Eight / Crowdfunder, 2015 (CC BY-NC-SA 4.0).